

L2 一手写你的第一个 Agent Loop (Beginner 核心)

目标：把 L1 那段伪代码变成**能跑的 Python**。学完你会亲手写出「LLM 决定调工具 → 你的代码执行 → 结果喂回 → 循环」的完整闭环。这是整门课**最重要的一节**，后面所有 内容都是往这个 loop 上加东西。

1. 回顾：从「一次调用」到「循环」

L1 的 `hello_llm.py` 是一问一答：发消息 → 拿回复 → 结束。agent 的不同只有两点，但它俩是灵魂：

1. **给模型配了「工具」**，模型可以选择「我要调用某个工具」而不是直接回答。
 2. **外面套了个 `while` 循环**：模型要调工具 → 我执行 → 把结果塞回 `messages` → 再问模型 → 直到模型说「我不需要工具了，这是最终答案」。
-

2. Tool use 的四步机制 (务必记牢)

这是本节的核心。所谓 function calling / tool use，就是这四步在循环里转：

你声明工具	告诉模型：我有哪些工具、每个工具叫什么、要什么参数
模型请求调用	模型不直接答，而是返回 "我要调查订单，参数 {id: 123}"
你的代码执行	你读到这个请求，真的去跑那个函数，拿到结果
结果喂回模型	把结果作为一条新消息塞回 <code>messages</code> ，再问模型一次 → 模型要么再调工具(回到)，要么给最终答案(结束)

再强调 L1 的铁律：模型永远只输出文字（第 一步只是输出一段结构化请求）。第 步真正 执行的是**你的代码**。模型和真实世界之间隔着你这一层。

3. 每一步对应的 API 写法 (DeepSeek / OpenAI 兼容)

声明工具：一个 JSON schema

```
tools = [
    {
        "type": "function",
        "function": {
            "name": "query_order",                # 工具名，模型靠它指名要调谁
            "description": "根据订单号查询订单状态，包括是否发货、预计送达时间",    # 关键！模型靠这句话判断啥时候
            "parameters": {                      # 参数的 JSON schema
                "type": "object",
                "properties": {
                    "order_id": {
                        "type": "string",
                        "description": "订单号，例如 A123"
                    }
                }
            },
            "required": ["order_id"],
        },
    },
]
```

```

    },
}
]

```

description 写得好不好，直接决定模型会不会在对的时机调用它——这是**提示工程**在 工具层的体现，后面会反复打磨。

调用时把工具传进去

```

response = client.chat.completions.create(
    model="deepseek-chat",
    messages=messages,
    tools=tools,          # ← 就多这一个参数
)
msg = response.choices[0].message

```

的结果：模型怎么表达「我要调工具」

- 如果模型决定调工具：msg.tool_calls 是一个**列表**（可能一次要调多个），每个元素有：
 - tool_call.id ——这次调用的唯一编号（第 一步要用它把结果对回去）
 - tool_call.function.name ——要调哪个工具，如"query_order"
 - tool_call.function.arguments ——参数，**是一段 JSON 字符串**，要用 json.loads() 解析成 dict
- 如果模型决定直接回答：msg.tool_calls 是 None，答案在 msg.content。

你的代码执行工具

```

import json

def query_order(order_id: str) -> str:
    # 真实项目里这里查数据库；现在先写死一个假数据练手
    fake = {"A123": " 已发货, 预计明天下午送达", "A999": " 仍在打包中"}
    return fake.get(order_id, " 查无此订单")

# 把「工具名」映射到「真正的 python 函数」
available_tools = {"query_order": query_order}

```

把结果喂回去（最容易写错的一步）

要往 messages 里追加**两样东西**：1. 模型那条「要调工具」的消息本身 (msg)；2. 每个工具的执行结果，角色是"tool"，并且**必须带上对应的 tool_call_id**，模型靠它知道「这个结果是回应我刚才哪次调用的」。

```

messages.append(msg)                                # 先把模型的「调用请求」记进去
for tool_call in msg.tool_calls:
    name = tool_call.function.name
    args = json.loads(tool_call.function.arguments)  # JSON 字符串 → dict
    result = available_tools[name](**args)          # 真正执行
    messages.append({
        "role": "tool",
        "tool_call_id": tool_call.id,              # ← 必须！把结果对回那次调用
    })

```

```

        "content": str(result),
    })

```

4. 完整 Loop 的骨架

把四步套进 while, 就是一个能跑的 agent:

```

messages = [系统提示, 用户问题]
while True:
    msg = 调用LLM(messages, tools)          #
    if msg.tool_calls:                       # 模型要调工具
        messages.append(msg)                # -1 记录调用请求
        for tc in msg.tool_calls:           # +-2 执行每个工具、把结果塞回
            执行并 append role=tool 的结果
            continue                         # 回到 while 顶, 再问模型
    else:                                    # 模型给最终答案了
        print(msg.content)
        break

```

盯住三个点: 1. **continue 让它回到循环顶**, 带着工具结果再问一次模型——这就是「转圈」。2. **必须加防护**: 万一模型陷入死循环一直调工具? 真实代码要加**最大轮数上限** (比如转 10 圈还没结束就强制退出), 否则可能烧钱烧到失控。3. **模型可能一次要调多个工具**, 所以第 步是个 for 循环遍历 msg.tool_calls。

5. 常见坑 (提前打预防针)

坑	症状	解法
忘了把 msg 本身 append 回去	API 报错: tool 消息找不到对应的 assistant 调用	先 append msg, 再 append 各 tool 结果
arguments 直接当 dict 用 tool 结果忘了 tool_call_id	TypeError, 它其实是 字符串 API 报错或模型对不上号	json.loads(arguments) 每条 tool 消息都带上对应 tool_call.id
没有轮数上限	偶发死循环、狂烧 token	while 里加计数器, 超过 N 轮强退

6. 本节小结

- Agent = 在一次性调用外面套一个 while 循环 + 给模型配 工具。
- Tool use 四步: 声明工具 → 模型请求 → 你执行 → 结果喂回, 在循环里转。
- arguments 是 JSON 字符串要 json.loads; tool 结果必须带 tool_call_id; 一定要加**最大轮数上限**。
- 模型只输出「调用请求」, **真正干活的永远是你的代码**。

作业: 你会亲手把这个 loop 写出来, 做一个能查订单的迷你客服 agent。